정규표현식

1. 정규표현식 기초 문법

정규표현식은 특정한 패턴을 가진 문자열을 찾기 위해 약속된 기호를 사용함

1.1 문자 클래스 및 범위 `[]`

- `[abc]`: a, b, c 중 하나와 매치 (예: `before`의 'b'는 매치되나 `dude`는 매치 안 됨)
- `[a-z]`, `[A-Z]`: 소문자 또는 대문자 알파벳 범위
- `[0-9]`, `[가-힣]`: 모든 숫자 또는 모든 한글 범위
- `[^0-9]`: 숫자가 아닌 모든 문자(특수문자, 공백 등)를 의미 (내부의 `^`는 부정을 의미)

1.2 주요 메타 문자 및 약어

기호	설명	동일한 표현
`.`	`\n`을 제외한 모든 문자 하나 (예: `a.b` → a와 b 사이 아무 문자)	-
`\d`	임의의 숫자 하나	`[0-9]`
`\D`	숫자가 아닌 문자	`[^0-9]`
`\s`	화이트스페이스 (공백, `\n`, `\t`, `\r` 등)	`[ \t\n\r\f\v]`
`\S`	공백이 아닌 모든 문자	`[^ \t\n\r\f\v]`
`\w`	단어 구성 문자 (알파벳 + 숫자 + 언더바)	`[a-zA-Z0-9_]`
`\W`	단어 구성 문자가 아닌 것	`[^a-zA-Z0-9_]`

1.3 반복 및 조건 수량자

- `*`: 0번 이상 반복 (예: `ca*t` → ct, cat, caat...)
- `+`: 1번 이상 반복 (최소 1회 필요)
- `?`: 0번 또는 1번 (있어도 되고 없어도 됨)
- `{n, m}`: n번부터 m번까지 반복 횟수 지정

- `|`: OR 연산 (예: `a|b` → a 또는 b)

1.4 위치 지정자

- `^` / `\A`: 문자열의 시작 부분과 매칭 (`\A`는 멀티라인 모드에서도 전체 문자열의 처음만 매칭)
- `$` / `\Z`: 문자열의 끝 부분과 매칭
- `\b`: 단어 구분자 (보통 공백과 매칭). 파이썬에서는 백스페이스와 구분하기 위해 `r'\b'` 형태로 사용 필수
- `\B`: 단어 구분자가 아닌 문자와 매칭

2. 파이썬 re 모듈 활용법

파이썬에서는 re 모듈을 통해 정규표현식을 제어함

2.1 주요 컴파일 메서드

```
import re
p = re.compile('[a-z]+') # 패턴 객체 생성

m = p.match('python')    # 문자열의 처음부터 매치되는지 확인
s = p.search('3 python') # 문자열 전체를 검색 (숫자가 앞에 있어도 매치됨)
f = p.findall('Life is too short') # 매치되는 모든 문자열을 '리스트'로 반환
i = p.finditer('Life is too short') # 매치되는 객체를 '반복 가능한 객체(iterator)'로 반환
```

2.2 Match 객체의 주요 메서드

매칭된 결과(`m` 또는 `s`)에서 상세 정보를 추출할 때 사용함

- `group()`: 매치된 문자열 전체를 반환

- `start()` / `end()`: 매치된 문자열의 시작 및 끝 인덱스 반환
- `span()`: (시작, 끝) 위치를 튜플로 반환

2.3 컴파일 옵션 (Flags)

- `re.DOTALL` (또는 `re.S`): `.` 문자가 줄바꿈(`\n`) 문자까지 포함하여 매치하게 함
- `re.IGNORECASE` (또는 `re.I`): 대소문자 구분 없이 매치
- `re.MULTILINE` (또는 `re.M`): `^`, `$` 메타 문자를 각 줄의 처음과 끝으로 적용
- `re.VERBOSE` (또는 `re.V`): 정규식 내에 주석 및 공백을 허용하여 가독성을 높임 (복잡한 패턴 설계 시 유용)

3. 고급 기능: 그루핑 및 탐색

3.1 그루핑 (Grouping) 및 재참조

특정 부분을 묶거나 이름을 붙여 관리 가능

```
# 그룹화 및 이름 붙이기 (?P<이름>)
p = re.compile(r"(?P<name>\w+)\s+(?P<phone>\d+[-]\d+[-]\d+)")
m = p.search("park 010-1234-1234")

print(m.group("name")) # 이름으로 호출 가능: 'park'
print(p.sub("\g<2> \g<1>", "park 010-1234-1234")) # 그룹 순서 바꾸어 치환 가능

# 그룹 재참조: 동일한 단어가 반복되는지 확인
p = re.compile(r'(?P<word>\b\w+)\s+(?P=word)')
p.search('Paris in the the spring').group() # 'the the' 매칭
```

3.2 전방 탐색

특정 조건을 확인하되, 결과값에는 포함시키지 않을 때 사용함

- 긍정적 전방 탐색 (`(?=...)`): 조건이 일치하는 경우만 검색

- 부정적 전방 탐색 (`(?!...)`): 조건이 일치하지 않는 경우만 검색

```
p = re.compile(".+(?=:)") # ':'이 뒤에 오는 패턴을 찾되 ':'은 결과에서 제외
m = p.search("http://google.com") # 'http'만 반환
```

4. 활용 사례

4.1 이메일 형식 검증

`^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$`

- 영문/숫자/특수문자로 시작해서 `@`가 있고 도메인 주소와 2글자 이상의 최상위 도메인이 오는 형식을 검사

4.2 휴대폰 번호 형식 검증

`^010-\d{3,4}-\d{4}$`

- 010으로 시작하며 중간에 3~4자리 숫자, 마지막에 4자리 숫자가 하이픈으로 연결된 패턴 검사

4.3 비밀번호 형식 검증

`^(?=.[A-Za-z])(?=.\d)[A-Za-z\d]{8,16}$`

- 전방 탐색을 사용해 영문과 숫자가 최소 하나씩 포함되었는지 확인하며 길이 제한